JAVA OOP LABORATORY ACTIVITY

CAMPUS CAFE ORDERING SYSTEM

WRITTEN REPORT

Members:	VILLAHERMOSA, PEPE JR. P. BANZON, NICK
Section:	E
Course:	BSINT
Date:	November 10, 2026
Programming Language:	Java

1. Introduction

The Campus Cafe Ordering System is a Java program for taking and calculating cafe orders. A customer can order up to three items. The items can be drinks, pastries, or meals. The program calculates the price of each item and the total amount to pay. It can also give a 10% discount to student customers, accept a voucher, and add loyalty points.

2. Java Classes Used

MenuItem. This is the parent class. It stores common information such as the item name, price, and code.

Beverage. This class is for drinks. The price can change depending on the drink size.

Pastry. This class is for baked food. It adds an extra charge when the pastry is warmed.

Meal. This class is for meals. It can add a charge for upsizing and extra sides.

Customer. This class stores the customer's name, student status, and loyalty points.

Order. This class adds items, calculates the subtotal and total, applies discounts or vouchers, and prints the receipt.

CafeApp. This is the main class. It creates sample customers and orders, then runs the program.

3. OOP Concepts Used

Classes and Objects. A class is a plan for creating an object. The program uses seven classes to organize the cafe system.

Encapsulation. Some data is private. Getters and setters are used to access or change data safely. Setters also check invalid values.

Inheritance. Beverage, Pastry, and Meal get common details from the MenuItem parent class.

Abstraction. MenuItem is an abstract class. It requires each child class to create its own computePrice() method.

Polymorphism. The child classes have their own versions of computePrice(). Some methods are also overloaded, such as addPoints() and computeTotal().

Constructors. Constructors give starting values when objects are created. The program uses default and parameterized constructors.

4. Program Output and Results

The program shows two sample orders. In the first order, Ana is a student. Her three items cost ■317.00 before the discount. The 10% student discount is 31.70, so she pays 285.30 and receives 5 loyalty points. In the second order, Ben's items cost 225.00. After using a 20.00 voucher, he pays 205.00 and receives 4 loyalty points.

```
Order is full. Cannot add Cookie [P-02].

Order #1001 - Ana Reyes (Student)
```

```
  Iced Latte [B-01] Size: L                   120.00
  Ensaymada [P-01] (Warmed)                    55.00
  Chicken Adobo [M-01] Upsized, 1 sides       142.00
  Subtotal: 317.00
  Student discount (10%): -31.70
  TOTAL: 285.30
  Loyalty points: 5 Error: base price

cannot be negative.

Order #1002 - Ben Cruz
  Hot Chocolate [B-02] Size: M                 95.00
  Beef Tapa [M-02] Regular, 0 sides           130.00
  Subtotal: 225.00
  Voucher: -20.00
  TOTAL: 205.00
  Loyalty points: 4
```

5. Challenges and Solutions

One challenge was making the seven classes work together. I used MenuItem as the parent class for Beverage, Pastry, and Meal. Another challenge was checking wrong values, such as a negative price or an invalid drink size. I used setters with conditions to prevent these values. I also made the Order class stop customers from adding more than three items. These checks help the program work correctly.

6. Conclusion

This activity helped me understand how Java classes and objects work together. I learned how to use constructors, encapsulation, inheritance, abstraction, and polymorphism. The program can calculate orders, apply discounts and vouchers, and give loyalty points. It shows how OOP can be used to make a simple cafe ordering system.

7. Appendix

Appendix A – Source Code

The project contains seven Java source files:

1. MenuItem.java
2. Beverage.java
3. Pastry.java
4. Meal.java

5. Customer.java
6. Order.java
7. CafeApp.java

Appendix B – Program Screenshots

Figure 1. Seven Java Classes

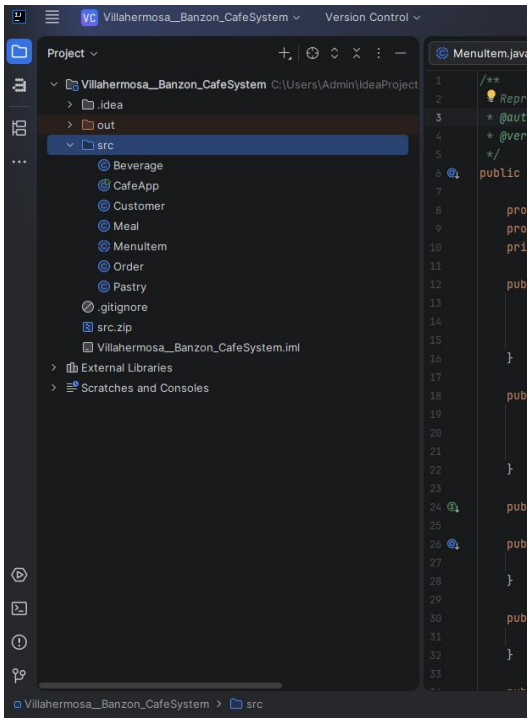


Figure 2. Java Source Code

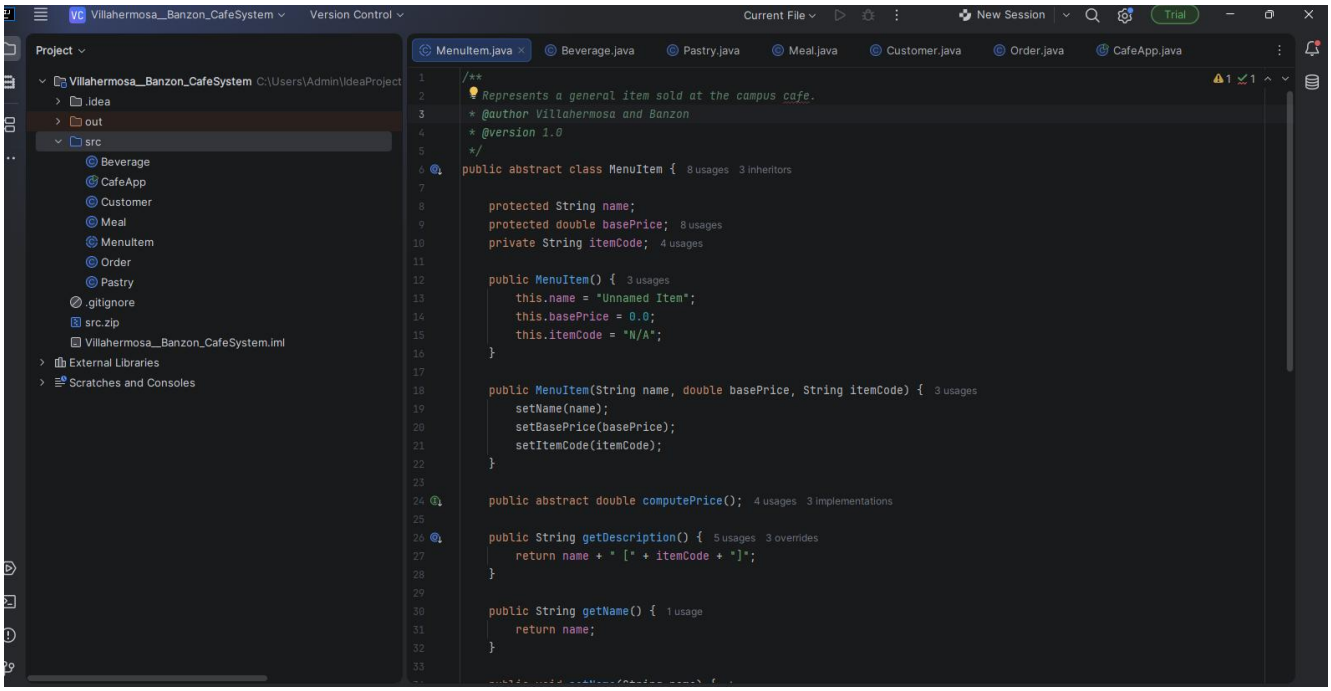
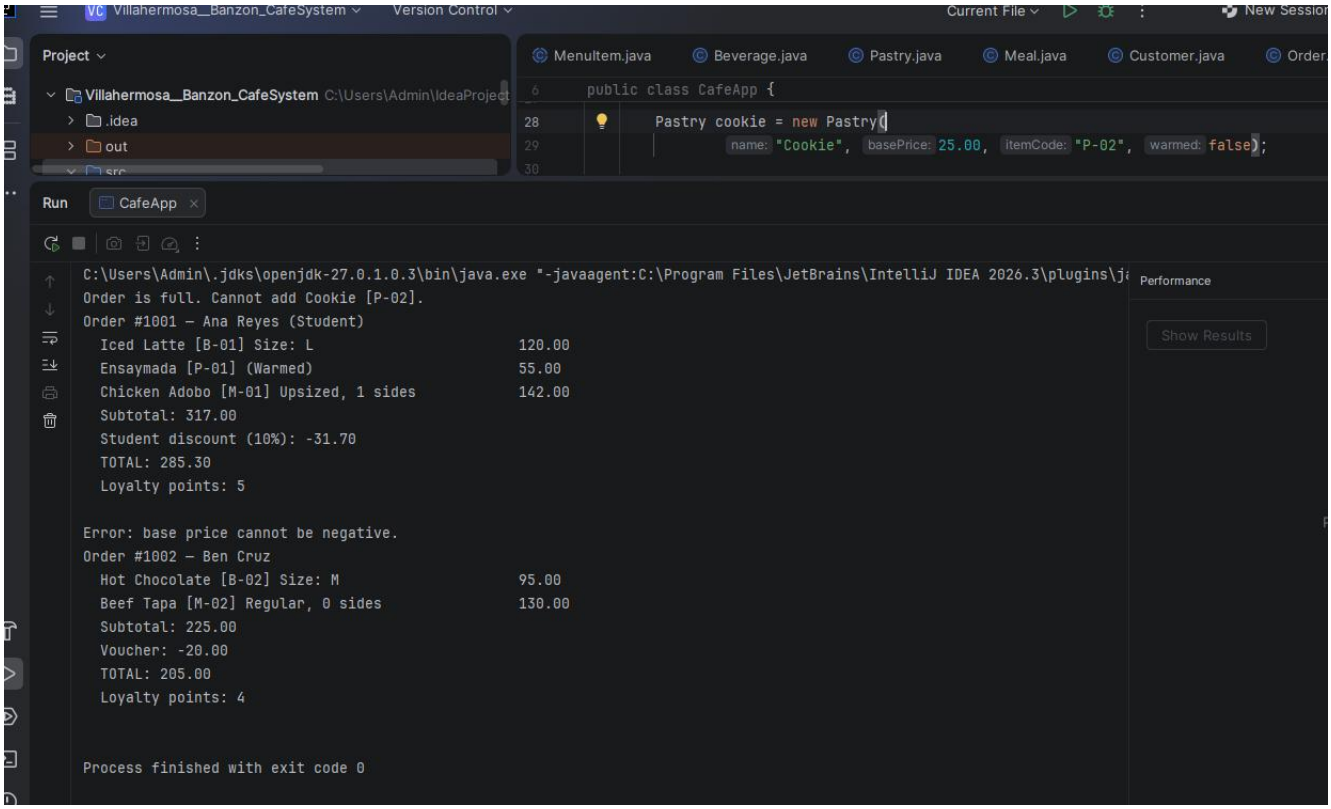


Figure 3. Program Output


Villahermosa__Banzon_CafeSystem
Version Control
Current File
Project
Villahermosa__Banzon_CafeSystem C:\Users\Admin\IdeaProject
.idea
out
MenuItem.java
Beverage.java
Pastry.java
Meal.java
Customer.java
public class CafeApp {
Pastry cookie = new Pastry(
name: "Cookie", basePrice: 25.00, itemCode: "P-02", warmed: false);
Run
CafeApp
C:\Users\Admin\.jdks\openjdk-27.0.1.0.3\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.3\plugins\j
Order is full. Cannot add Cookie [P-02].
Order #1001 — Ana Reyes (Student)
Iced Latte [B-01] Size: L 120.00
Ensaymada [P-01] (Warmed) 55.00
Chicken Adobo [M-01] Upsized, 1 sides 142.00
Subtotal: 317.00
Student discount (10%): -31.70
TOTAL: 285.30
Loyalty points: 5
Error: base price cannot be negative.
Order #1002 — Ben Cruz
Hot Chocolate [B-02] Size: M 95.00
Beef Tapa [M-02] Regular, 0 sides 130.00
Subtotal: 225.00
Voucher: -20.00
TOTAL: 205.00
Loyalty points: 4
Process finished with exit code 0
Performance
Show Results